

Practical -1

Aim:

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Objective:

A Sorting Algorithm is used to rearrange a given array or list of elements in an order. For example, a given array [10, 20, 5, 2] becomes [2, 5, 10, 20] after sorting in increasing order and becomes [20, 10, 5, 2] after sorting in decreasing order.

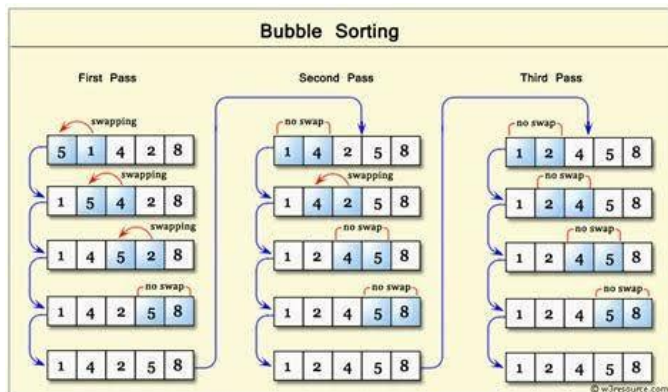
- There exist different sorting algorithms for different different types of inputs, for example a binary array, a character array, an array with a large range of values or an array with many duplicates or a small vs large array.
- The algorithms may also differ according to output requirements. For example, stable sorting (or maintains original order of equal elements) or not stable.
- Sorting is provided in library implementation of most of the programming languages. These sorting functions typically are general purpose functions with flexibility of providing the expected sorting order (increasing or decreasing or by a specific key in case of objects).

Sorting Algorithms:

Bubble Sort:

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not efficient for large data sets as its average and worst-case time complexity are quite high.

- Sorts the array using multiple passes. After the first pass, the maximum goes to end (its correct position). Same way, after second pass, the second largest goes to second last position and so on.
- In every pass, process only those that have already not moved to correct position. After k passes, the largest k must have been moved to the last k positions.
- In a pass, we consider remaining elements and compare all adjacent and swap if larger element is before a smaller element. If we keep doing this, we get the largest (among the remaining elements) at its correct position.



CODE :

```
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    bool swapped;

    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }

        // If no two elements were swapped, then break
        if (!swapped)
            break;
    }
}
```

Complexity Analysis of Bubble Sort:

- **Time Complexity:** $O(n^2)$
- **Auxiliary Space:** $O(1)$

Please refer Complexity Analysis of Bubble Sort for details.

Advantages of Bubble Sort

- Bubble sort is easy to understand and implement.
- It does not require any additional memory space.
- It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.

Disadvantages of Bubble Sort

- Bubble sort has a time complexity of $O(n^2)$ which makes it very slow for large data sets.
- Bubble sort has almost no or limited real world applications. It is mostly used in academics to teach different ways of sorting.

Quick Sort

QuickSort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array. .

There are mainly three steps in the algorithm:

1. **Choose a Pivot:** Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).

2. Partition the Array: Rearrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right.
3. Recursively Call: Recursively apply the same process to the two partitioned sub-arrays.
4. Base Case: The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

Choice of Pivot

There are many different choices for picking pivots.

- Always pick the first (or last) element as a pivot. The below implementation picks the last element as pivot. The problem with this approach is it ends up in the worst case when array is already sorted.
- Pick a random element as a pivot. This is a preferred approach because it does not have a pattern for which the worst case happens.
- Pick the median element as pivot. This is an ideal approach in terms of time complexity as we can find median in linear time and the partition function will always divide the input array into two halves. But it takes more time on average as median finding has high constants.

CODE:

```
int partition(vector<int>& arr, int low, int high) {  
    // choose the pivot  
    int pivot = arr[high];  
  
    // index of smaller element and indicates  
    // the right position of pivot found so far  
    int i = low - 1;
```

```
// Traverse arr[low..high] and move all smaller
// elements on left side. Elements from low to
// i are smaller after every iteration
for (int j = low; j <= high - 1; j++) {
    if (arr[j] < pivot) {
        i++;
        swap(arr[i], arr[j]);
    }
}
// move pivot after smaller elements and
// return its position
swap(arr[i + 1], arr[high]);
return i + 1;
}
// the QuickSort function implementation
void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        // pi is the partition return index of pivot
        int pi = partition(arr, low, high);
        // recursion calls for smaller elements
        // and greater or equals elements
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Complexity Analysis of Quick Sort

Time Complexity:

- **Best Case:** ($\Omega(n \log n)$), Occurs when the pivot element divides the array into two equal halves.
- **Average Case** ($\theta(n \log n)$), On average, the pivot divides the array into two parts, but not necessarily equal.
- **Worst Case:** ($O(n^2)$), Occurs when the smallest or largest element is always chosen as the pivot (e.g., sorted arrays).

Auxiliary Space:

- **Worst-case scenario:** $O(n)$ due to unbalanced partitioning leading to a skewed recursion tree requiring a call stack of size $O(n)$.
- **Best-case scenario:** $O(\log n)$ as a result of balanced partitioning leading to a balanced recursion tree with a call stack of size $O(\log n)$.

Advantages of Quick Sort

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It has a low overhead, as it only requires a small amount of memory to function.
- It is Cache Friendly as we work on the same array to sort and do not copy data to any auxiliary array.
- Fastest general purpose algorithm for large data when stability is not required.
- It is [tail recursive](#) and hence all the [tail call optimization](#) can be done.

Disadvantages of Quick Sort

- It has a worst-case time complexity of $O(n^2)$, which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.
- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions).

Applications of Quick Sort

- Sorting large datasets efficiently in memory.
- Used in library sort functions (like C++ `std::sort` and Java `Arrays.sort` for primitives).
- Arranging records in databases for faster searching.
- Preprocessing step in algorithms requiring sorted input (e.g., binary search, two-pointer techniques).
- Finding the kth smallest/largest element using Quickselect (a variant of quicksort).
- Sorting arrays of objects based on multiple keys (custom comparators).
- Data compression algorithms (like Huffman coding preprocessing).
- Graphics and computational geometry (e.g., convex hull algorithms).

Selection Sort:

Selection Sort is a comparison-based sorting algorithm. It sorts by repeatedly selecting the smallest (or largest) element from the unsorted portion and swapping it with the first unsorted element.

1. Find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.
2. Then find the smallest among remaining elements (or second smallest) and swap it with the second element.

3. We keep doing this until we get all elements moved to correct position.

Code:

```
void selectionSort(vector<int> &arr) {  
    int n = arr.size();  
    for (int i = 0; i < n - 1; ++i) {  
        int min_idx = i;  
        for (int j = i + 1; j < n; ++j) {  
            if (arr[j] < arr[min_idx]) {  
                min_idx = j;  
            }  
        }  
        swap(arr[i], arr[min_idx]);  
    }  
}
```

Complexity Analysis of Selection Sort

Time Complexity: $O(n^2)$, as there are two nested loops:

- One loop to select an element of Array one by one = $O(n)$
- Another loop to compare that element with every other Array element = $O(n)$
- Therefore overall complexity = $O(n) * O(n) = O(n*n) = O(n^2)$

Auxiliary Space: $O(1)$ as the only extra memory used is for temporary variables.

Advantages of Selection Sort

- Easy to understand and implement, making it ideal for teaching basic sorting concepts.
- Requires only a constant $O(1)$ extra memory space.

- It requires less number of swaps (or memory writes) compared to many other standard algorithms. Only [cycle sort](#) beats it in terms of memory writes. Therefore it can be simple algorithm choice when memory writes are costly.

Disadvantages of the Selection Sort

- Selection sort has a time complexity of $O(n^2)$ makes it slower compared to algorithms like [Quick Sort](#) or [Merge Sort](#).
- Does not maintain the relative order of equal elements which means it is not stable.

Applications of Selection Sort

- Perfect for teaching fundamental sorting mechanisms and algorithm design.
- Suitable for small lists where the overhead of more complex algorithms isn't justified and memory writing is costly as it requires less memory writes compared to other standard sorting algorithms.
- [Heap Sort](#) algorithm is based on Selection Sort.

Merge Sort:

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the Divide and Conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Here's a step-by-step explanation of how merge sort works:

1. Divide: Divide the list or array recursively into two halves until it can no more be divided.
2. Conquer: Each subarray is sorted individually using the merge sort algorithm.
3. Merge: The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

CODE :

```
void merge(vector<int>& arr, int left,
           int mid, int right){

    int n1 = mid - left + 1;

    int n2 = right - mid;

    // Create temp vectors
    vector<int> L(n1), R(n2);

    // Copy data to temp vectors L[] and R[]
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0;
    int k = left;

    // Merge the temp vectors back
    // into arr[left..right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
```

```
    arr[k] = L[i];  
    i++;  
}  
else {  
    arr[k] = R[j];  
    j++;  
}  
k++;  
}  
  
// Copy the remaining elements of L[],  
// if there are any  
while (i < n1) {  
    arr[k] = L[i];  
    i++;  
    k++;  
}  
  
// Copy the remaining elements of R[],  
// if there are any  
while (j < n2) {  
    arr[k] = R[j];  
    j++;
```

```
        k++;  
    }  
}  
  
// begin is for left index and end is right index  
// of the sub-array of arr to be sorted  
void mergeSort(vector<int>& arr, int left, int right){  
  
    if (left >= right)  
        return;  
  
    int mid = left + (right - left) / 2;  
    mergeSort(arr, left, mid);  
    mergeSort(arr, mid + 1, right);  
    merge(arr, left, mid, right);  
}
```

Recurrence Relation of Merge Sort

The recurrence relation of merge sort is:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1 \end{cases}$$

- $T(n)$ Represents the total time taken by the algorithm to sort an array of size n .
- $2T(n/2)$ represents time taken by the algorithm to recursively sort the two halves of the array. Since each half has $n/2$ elements, we have two recursive calls with input size as $(n/2)$.
- $O(n)$ represents the time taken to merge the two sorted halves

Complexity Analysis of Merge Sort

Time Complexity:

- Best Case: $O(n \log n)$, When the array is already sorted or nearly sorted.
- Average Case: $O(n \log n)$, When the array is randomly ordered.
- Worst Case: $O(n \log n)$, When the array is sorted in reverse order.

Auxiliary Space: $O(n)$, Additional space is required for the temporary array used during merging.

Applications of Merge Sort:

- Sorting large datasets
- [External sorting](#) (when the dataset is too large to fit in memory)
- Used to solve problems like [Inversion counting](#), [Count Smaller on Right](#) & [Surpasser Count](#)
- Merge Sort and its variations are used in library methods of programming languages. Its variation [TimSort](#) is used in Python, Java Android and Swift. The main reason why it is preferred to sort non-primitive types is stability which is not there in QuickSort. [Arrays.sort in Java](#) uses QuickSort while [Collections.sort](#) uses MergeSort.
- It is a preferred algorithm for sorting Linked lists.
- It can be easily parallelized as we can independently sort subarrays and then merge.
- The merge function of merge sort to efficiently solve the problems like [union and intersection of two sorted arrays](#).

Advantages and Disadvantages of Merge Sort

Advantages

- Stability : Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
- Guaranteed worst-case performance: Merge sort has a worst-case time complexity of $O(N \log N)$,

which means it performs well even on large datasets.

- Simple to implement: The divide-and-conquer approach is straightforward.
- Naturally Parallel : We independently merge subarrays that makes it suitable for parallel processing.

Disadvantages

- Not an in-place, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- Slower than QuickSort in general as QuickSort is more cache friendly because it works in-place

Insertion Sort Algorithm:

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is like sorting playing cards in your hands. You split the cards into two groups: the sorted cards and the unsorted cards. Then, you pick a card from the unsorted group and put it in the right place in the sorted group.

- Start with the second element as the first element is assumed to be sorted.
- Compare the second element with the first if the second is smaller then swap them.
- Move to the third element, compare it with the first two, and put it in its correct position
- Repeat until the entire array is sorted.

CODE :

```
/* Function to sort array using insertion sort */
```

```
void insertionSort(int arr[], int n)
```

```
{
```

```
    for (int i = 1; i < n; ++i) {
```

```
        int key = arr[i];
```

```
int j = i - 1;
```

```
/* Move elements of arr[0..i-1], that are  
greater than key, to one position ahead  
of their current position */
```

```
while (j >= 0 && arr[j] > key) {
```

```
    arr[j + 1] = arr[j];
```

```
    j = j - 1;
```

```
}
```

```
arr[j + 1] = key;
```

```
}
```

```
}
```

Complexity Analysis of Insertion Sort

Time Complexity

- Best case: $O(n)$, If the list is already sorted, where n is the number of elements in the list.
- Average case: $O(n^2)$, If the list is randomly ordered
- Worst case: $O(n^2)$, If the list is in reverse order

Space Complexity

- Auxiliary Space: $O(1)$, Insertion sort requires $O(1)$ additional space, making it a space-efficient sorting algorithm.

Please refer [Complexity Analysis of Insertion Sort](#) for details.

Advantages and Disadvantages of Insertion Sort

Advantages

- Simple and easy to implement.
- Stable sorting algorithm.
- Efficient for small lists and nearly sorted lists.
- Space-efficient as it is an in-place algorithm.
- Adaptive. the [number of inversions](#) is directly proportional to number of swaps. For example, no swapping happens for a sorted array and it takes $O(n)$ time only.

Disadvantages

- Inefficient for large lists.
- Not as efficient as other sorting algorithms (e.g., merge sort, quick sort) for most cases.

Applications of Insertion Sort

Insertion sort is commonly used in situations where:

- The list is small or nearly sorted.
- Simplicity and stability are important.
- Used as a subroutine in [Bucket Sort](#)
- Can be useful when array is already almost sorted (very few [inversions](#))
- Since Insertion sort is suitable for small sized arrays, it is used in [Hybrid Sorting algorithms](#) along with other efficient algorithms like Quick Sort and Merge Sort. When the subarray size becomes small, we switch to insertion sort in these recursive algorithms. For example [IntroSort](#) and [TimSort](#) use insertions sort.